SPOT LIGHT BIRIYANI - LINE-BY-LINE & METHOD-BY-METHOD CODE WALKTHROUGH

A Complete Architectural & Implementation Manual for Engineers, Architects & Operators

1. Executive Codebase Overview

The Spot Light Biriyani application is built using a modern, high-speed single-page application (SPA) architecture designed to run reliably both online and completely offline.

Tech Stack Breakdown:

- **Framework & Bundler:** React 19 + Vite 6
- **Routing Engine:** TanStack Router (`@tanstack/react-router`) with full code-splitting and type safety.
- **Styling Architecture:** Tailwind CSS v4 with OKLCH theme tokens and custom glassmorphism.
- **State Management & Persistence:** Zero-dependency Reactive LocalStorage Event Store with `window.dispatchEvent(new Event("storage"))` multi-tab synchronization.
- **Audio Synthesizer:** Native Web Audio API with dual-oscillator acoustic modeling (no external MP3/WAV assets needed).
- **Thermal Printing:** Native browser print API styled to exact 80mm thermal receipt specifications.

2. Core State Store: `src/lib/pos-store.ts`

2.1 Type Definitions

```
export interface Branch { id: string; name: string; landmark: string; phone: string; dumBatchTime:
 string; }
export interface MenuItem { id: string; name: string; category: Category; price: number; onlinePri
ce: number; cost: number; active: boolean; }
export interface StockItem { id: string; name: string; unit: string; qty: number; reorderLevel: nu
mber; costPerUnit: number; }
export interface SaleLine { itemId: string; name: string; qty: number; price: number; cost: number
; }
export interface Sale { id: string; token: number; date: string; branchId: string; channel: Channe
lId; lines: SaleLine[]; subtotal: number; discount: number; net: number; foodCost: number; commiss
ion: number; profit: number; payment: "cash" | "upi" | "card" | "online"; orderNote?: string; }
```

2.2 Method: `buildSale(draft)`

- **Purpose:** Pure business logic function that transforms raw cart selections into a finalized, audited financial record.
- **Line-by-Line Breakdown:**

1. Computes `subtotal`: `draft.lines.reduce((acc, l) => acc + l.qty * l.price, 0)`
2. Computes `foodCost`: `draft.lines.reduce((acc, l) => acc + l.qty * l.cost, 0)`
3. Applies `net`: `Math.max(0, subtotal - draft.discount)`
4. Calculates `commission`: `channel.commissionPct > 0 ? (net * channel.commissionPct) / 100 : 0`
5. Computes True Profit: `profit = net - foodCost - commission`
6. Assigns Daily Token Number: Filters existing sales for today's date and branch, returning `todaySales.length + 1`.

2.3 Method: `useDB()` & `setDB()`

• **Method `useDB()`:** Custom React hook that subscribes to local storage updates using `useEffect` and `addEventListener("storage")`. Any transaction executed on Tab A instantly triggers a state re-render on Tab B.

• **Method `setDB(updater)`:** Mutates the database safely in memory, saves to `localStorage.setItem("spotlight_pos_db", JSON.stringify(db))`, and fires a custom window event for zero-latency reactive updates.

---

3. POS Billing Terminal: `src/routes/app.billing.tsx`

3.1 State Hooks

• `cat`: Active category tab (`biryani`, `starter`, `kushka`, `breadhalwa`, `drink`).

• `lines`: Array of dishes in the current order ticket (`SaleLine[]`).

• `channel`: Active sales channel (`counter`, `swiggy`, `zomato`, `takeaway`).

• `discount`: Manual discount in Rupees.

• `payment`: Selected payment mode (`cash`, `upi`, `card`, `online`).

• `cashTendered`: Cash handed by customer (`number | ""`).

• `orderNote`: Kitchen instructions (e.g., "Less spicy, extra ennai kathirikai").

• `soundStyle`: User preference for billing chime (`kaching`, `triad`, `click`, `mute`).

3.2 Method: `step(itemId, delta)`

• **Purpose:** Increments or decrements dish quantity directly from either the menu cards or the order cart summary.

• **Logic:** Maps over `lines`. If `delta === +1`, increments `qty`. If `delta === -1` and `qty` reaches `0`, the dish is automatically filtered out of the cart.

3.3 Method: `addCustomItem()`

• **Purpose:** Handles off-menu items, catering requests, or special price adjustments requested by customers.

• **Logic:** Assigns an ephemeral ID `custom-${Date.now()}`, validates positive price, sets a realistic default food cost (45%), and appends the item to the bill ticket.

3.4 Method: `playChime(type, customStyle)`

• **Purpose:** Synthesizes realistic hardware sound effects using the browser's native `AudioContext` without loading external audio files.

• **Frequency Analysis:**

- **Ka-Ching:** Plays high metallic register bell (B5 at 987Hz ramping to E6 at 1318Hz) layered with resonant overtone (G6 at 1568Hz to C7 at 2093Hz) and a coin-strike pulse at 2637Hz.

- **Triad Chime:** Modern Apple Pay / GPay warm harmonic triad playing C5 (523Hz), E5 (659Hz), and G5 (784Hz) with gentle 350ms exponential envelope decay.

- **Print Feed:** Dual soft sine/triangle pulses at 320Hz and 480Hz fading in 100ms, mimicking a precision thermal receipt cutter engaging.

3.5 Method: `handlePrintThermalReceipt()`

• **Purpose:** Generates a pixel-perfect 80mm ESC/POS formatted receipt and sends it directly to USB/Bluetooth thermal printers.

• **Data Rendered:**

- Restaurant Header, Address & FSSAI License Number
- Branch Name & Cashier Terminal ID
- Date, Time & Token Number (e.g. `TOKEN #042`)
- Order Channel: Counter Dine-in / Swiggy / Zomato
- Line Items: Qty, Name, Rate, Line Total
- Subtotal, Discounts, Net Amount
- Payment Method: Cash / UPI / Card
- Cash Tendered by Customer & Return Change Due
- Kitchen Preparation Notes
- Barcode / QR Code placeholder for feedback

---

4. Stock & Raw Material Ledger: `src/routes/app.stock.tsx`

4.1 Data Architecture

Tracks 11 fundamental Biriyani raw ingredients:

1. Basmati Rice (kg)
2. Fresh Chicken (kg)
3. Fresh Mutton (kg)
4. Sambar / Red Onions (kg)
5. Country Tomatoes (kg)
6. Refined Sunflower Oil (L)
7. Country Ghee (L)
8. Signature Handi Biriyani Masala (kg)
9. Thick Farm Curd (kg)
10. Premium Thermal Packing Boxes (pcs)
11. Commercial LPG Cylinders (pcs)

4.2 Method: `applyMove()`

- **Purpose:** Records stock movements (Purchase In, Kitchen Dum Consumption Out, or Wastage/Spoilage).
- **Logic:** Updates item quantity `item.qty = Math.max(0, item.qty + (form.type === "in" ? qty : -qty))`. Adds a permanent timestamped movement record to `d.moves`.
- **Reorder Level Alarm:** Compares current stock with `s.reorderLevel`. When stock falls below threshold, renders a bold amber/red alert pill.

---

5. Menu Engineering & Plate Costs: `src/routes/app.items.tsx`

5.1 Financial Formulae

- **Plate Cost (Food Cost):** Actual cost of raw meat, rice, spices, oil, curd, and gas required to produce one single plate.

- **Counter Margin:** `margin = counterPrice - foodCost`
- **Margin Percentage:** `(margin / counterPrice) * 100`
- **Online Channel Markup:** `onlinePrice = counterPrice * 1.28` (accounts for 25% platform commission + 3% payment gateway charges).

5.2 Dynamic In-Line Editing

The menu table enables instant inline editing of Counter Price, Online Price, and Plate Cost. When prices are edited, profit reports and forecast calculations update instantaneously across all connected screens.

---

6. Financial Reports & Forecasting: `src/routes/app.reports.tsx`

6.1 Metrics Computed

- **Total Revenue (Net):** Sum of all completed bills for the active branch.
- **Total Food Cost:** Sum of recipe ingredient costs for all dishes sold.
- **Aggregator Commissions:** Total fees paid to Swiggy and Zomato.
- **True Net Profit:** Real money remaining in the bank after food cost and commissions.
- **Gross Margin Percentage:** `(True Net Profit / Net Sales) * 100`.

6.2 Dum Batch Predictor Engine

Uses historical sales velocity to calculate how many handis (batches of 40 plates) should be prepared for the upcoming lunch (11:30 AM) and dinner (6:00 PM) shifts to prevent stockouts and minimize wastage.

---

7. Interactive Chennai Kitchens Radar: `src/routes/branches.tsx`

7.1 Coordinate Radar Projection

The map renders an authentic South Chennai coastal radar visual with the Bay of Bengal coastline:

- **Tharamani Flagship Kitchen:** `top: 54%, left: 62%` (OMR / Kazhikundram / Tidel Park corridor)
- **Adyar Beach Kitchen:** `top: 38%, left: 70%` (LB Road / Besant Nagar beach corner)
- **Velachery Delivery Hub:** `top: 62%, left: 46%` (100 Feet Road / Bypass junction)

7.2 Interactive State

Tapping any pin activates its pulse animation, loads its real-time dum handi status ("14 Plates Left"), shows direct kitchen phone numbers, and provides a 1-tap Google Maps navigation intent.

---

8. Theme System & Light/Dark Engine: `src/styles.css`

8.1 Dual-Theme Architecture

- **Toast Dark Mode (Default):** Charcoal background (`#16120e`), warm ember accents (`#ea580c`), dark cards (`#241f1a`). Designed for low-light restaurant counters and evening operations.

• **Square Daylight Mode:** Crisp white cards (`#ffffff`), soft slate borders (`#e2e8f0`), deep charcoal text (`#0f172a`), and high-contrast inputs. Designed for high-glare daytime counters.

• **State Persistence:** Preference is saved to `localStorage.getItem("spotlight_pos_theme")` and remembered across sessions.

---

9. Summary & Deployment Readiness

This codebase contains zero external database dependencies for basic operation, allowing instant zero-cost deployment to Vercel, Netlify, or local offline cash registers with 100% data reliability.